РОЗДІЛ 1

АНАЛІЗ ЗАДАЧІ TEXT-TO-SQL, НАБОРІВ ДАНИХ І МЕТОДІВ АДАПТАЦІЇ МОВНИХ МОДЕЛЕЙ

У розділі зібрано постановку задачі перетворення природномовного запиту на SQL, еталонні й навчальні набори даних, метрики оцінювання, способи адаптації мовних моделей і наявні засоби контролю якості анотацій. Огляд спирається на першоджерела наборів і методів, на рецензовані огляди застосування великих мовних моделей до Text-to-SQL [1] і низькорангової адаптації [2], а також на препринт огляду етапів LLM-систем Text-to-SQL [3]. Власні методи й числа аудитів у цей розділ не входять.

1.1 Задача Text-to-SQL та розвиток підходів до її розв'язання

Для побудови SQL за питанням користувача потрібно врахувати зміст питання та структуру бази даних. Перед розглядом методів розв'язання цієї задачі визначимо її вхідні дані, очікуваний результат і вимоги до правильності запиту.

1.1.1 Постановка та формалізація задачі Text-to-SQL

Зростання обсягів даних у базах даних і аналітичних сховищах підвищує потребу в засобах їх пошуку та аналізу. Водночас отримання потрібної вибірки зазвичай потребує володіння мовою SQL, що обмежує доступ до даних для користувачів без відповідної технічної підготовки. Подолання цього бар'єра зумовило розвиток засобів, які дають змогу формулювати інформаційну потребу природною мовою [3].

Задача Text-to-SQL полягає в автоматичному перетворенні питання природною мовою на SQL-запит до заданої реляційної бази даних. Виконання цього запиту системою керування базами даних (СКБД) має надати відповідь на поставлене питання. Самої синтаксичної коректності запиту недостатньо. Він має звертатися до потрібних таблиць і колонок. Умови відбору, з'єднання, агрегування та інші операції мають відповідати змісту питання [4, 5].

У термінах оброблення природної мови Text-to-SQL належить до се-

мантичного парсингу [4], оскільки питання природною мовою відображають у формальне представлення його змісту. У цій задачі таким представленням є SQL-запит. Далі розглядається саме побудова такого запиту. Повний природномовний інтерфейс до бази даних є ширшим і може додатково підтримувати діалогове уточнення питання та подання результату користувачеві [6].

Позначимо питання природною мовою через q, а схему цільової бази даних через S. Схема містить назви таблиць і колонок, типи даних, первинні та зовнішні ключі й інші оголошені обмеження. Через D позначимо поточний стан бази, тобто вміст її таблиць. За незмінної схеми цей вміст може змінюватися. Відповідно, той самий запит може повертати різні результати.

У базовій постановці вхідними даними є питання q та схема S [4]. Для інтерпретації питання можуть знадобитися й додаткові відомості. До них належать описи колонок, значення з бази, предметні знання та правила обчислення показників [5]. Позначимо ці відомості через E (evidence). За їх відсутності $E = \varnothing$.

З урахуванням введених позначень запишемо перетворення Text-to-SQL:

$$(q, S, E) \xrightarrow{\text{Text-to-SQL}} y, \tag{1.1}$$

де y — SQL-запит у заданому діалекті δ. Потрібно побудувати запит, синтаксично коректний у цьому діалекті та узгоджений зі схемою S. Його зміст має відповідати питанню q з урахуванням E.

Для отримання відповіді SQL-запит y виконують у СКБД на поточному стані бази D. Результат виконання позначимо через

$$R = \text{Exec}_\delta(y, D). \tag{1.2}$$

Оператор Exec_δ враховує правила заданого діалекту δ. Результат R визначений за умови успішного виконання. Помилку виконання слід відрізняти від порожнього результату. Наприклад, коректний запит на відбір рядків поверне порожній результат, якщо жодний рядок не задовольняє заданої умови.

Розглянемо питання «Як звати співаків, старших за 30 років?» (рис. 1.1). Таблиця `singer` містить колонку `name` з іменами та колонку `age` з віком у роках. Слово «співаків» зіставляється з таблицею `singer`, а «як звати» — з колонкою `name`. Фрагмент «старших за 30 років» задає умову `age > 30`. Ви-

конання запиту повертає імена співаків, які задовольняють цю умову. Записи з віком рівно 30 років до результату не входять.

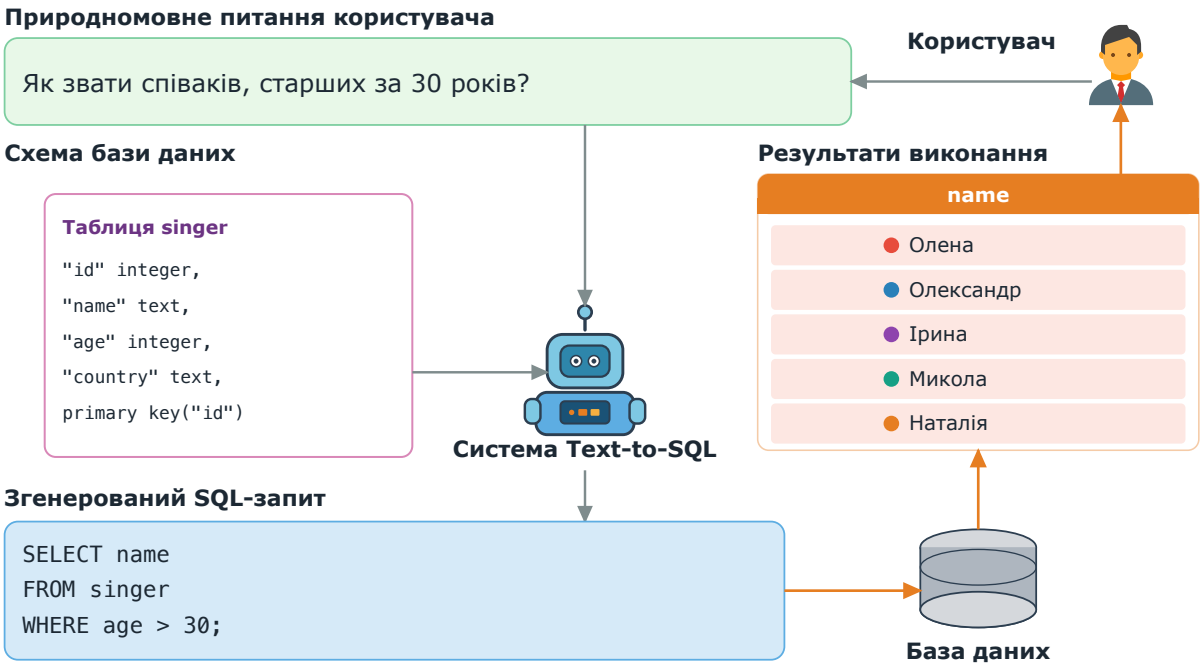


Рисунок 1.1 – Загальна схема природномовного доступу до бази даних із використанням систем Text-to-SQL

Семантична правильність запиту визначається його відповідністю змісту питання в контексті заданої бази. Різні за структурою SQL-запити можуть бути семантично еквівалентними. Водночас збіг результатів на одному стані D не доводить їхньої еквівалентності на інших станах. Наприклад, заміна `age > 30` на `age >= 30` у наведеному запиті змінює зміст умови. Проте обидва запити дадуть однакову відповідь, якщо в таблиці немає співаків віком рівно 30 років. Отже, слід розрізняти структурний збіг SQL, збіг результатів виконання та відповідність запиту питанню [1].

Основні труднощі Text-to-SQL пов’язані з неоднозначністю природної мови, зіставленням понять питання з таблицями та колонками бази даних і визначенням потрібних SQL-операцій. Складні питання можуть вимагати з’єднання таблиць, агрегування та вкладених запитів. Окремою проблемою є узагальнення на нові схеми й предметні області, не представлені в навчальних даних.

1.1.2 Еволюція підходів до Text-to-SQL

З погляду розвитку мовних моделей підходи до Text-to-SQL умовно поділяють на методи на основі правил, нейронні моделі, попередньо навчені мовні

моделі (PLM) та великі мовні моделі (LLM) [7]. Межі між цими групами не є чіткими, оскільки нові системи продовжують використовувати структурні обмеження й інші компоненти попередніх підходів. Водночас великі мовні моделі є масштабованим різновидом попередньо навчених моделей. Узагальнення розвитку підходів наведено на рис. 1.2.

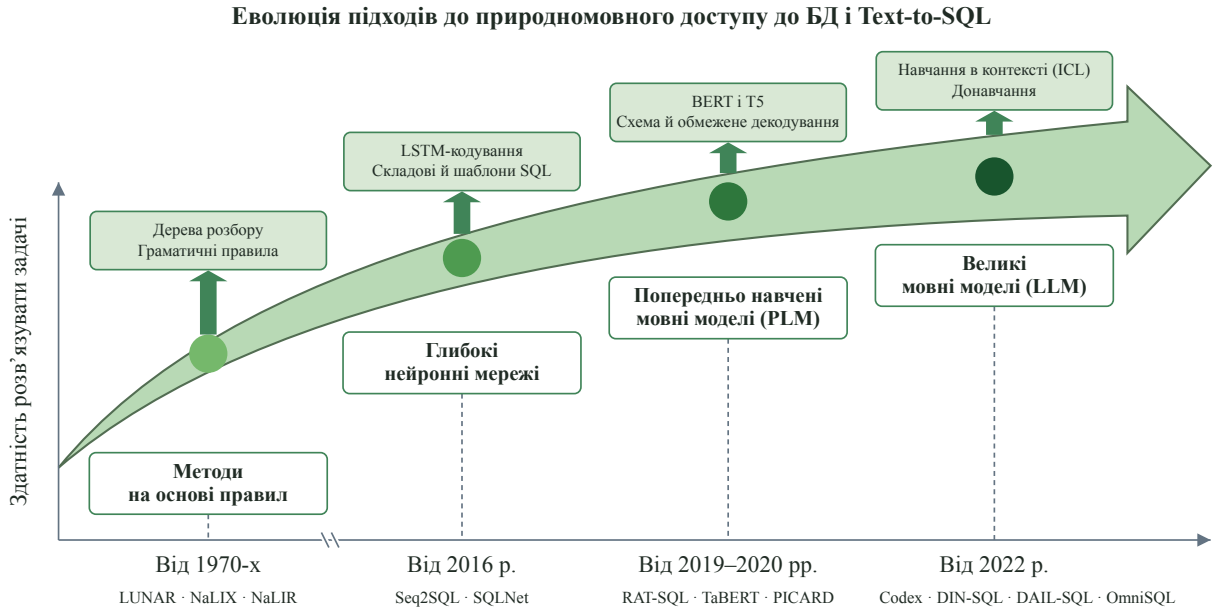


Рисунок 1.2 – Еволюція підходів до природномовного доступу до баз даних і Text-to-SQL

Методи на основі правил і шаблонів. Типовий процес у системах на основі правил передбачав лексичний і синтаксичний аналіз питання. Синтаксичну структуру питання часто подавали як дерево синтаксичного розбору. Семантичні правила зіставляли елементи цього дерева з таблицями, колонками, значеннями та операціями над даними. За результатами зіставлення система формувала запит за визначеними правилами й шаблонами. Конкретний формат залежав від системи. LUNAR будувала процедуру отримання відповіді [8], NaLIX — XQuery до XML-даних [9], а NaLIR — SQL-запит до реляційної бази [10].

Оскільки правила та відповідності задавали вручну, підтримка нових формулювань, елементів схеми й конструкцій запиту потребувала змін у системі. Це ускладнювало її супровід і перенесення на інші предметні області.

Глибокі нейронні мережі. Із середини 2010-х років для семантичного парсингу почали застосовувати архітектури «кодувальник–декодувальник» на основі рекурентних нейронних мереж, зокрема LSTM. Dong і Lapata застосували LSTM із механізмом уваги для породження логічних форм, хоча

їхня робота не була безпосередньо системою Text-to-SQL [11]. На відміну від систем на основі правил, такі моделі навчали встановлювати відповідності між природномовним питанням і формальним запитом на основі навчальних прикладів. У Text-to-SQL такий приклад пов'язував питання і схему бази даних з еталонним SQL-запитом. Seq2SQL [12] обирала агрегатну функцію й колонку для `SELECT`, а потім генерувала умови `WHERE`. Оскільки однакові умови можна записати в різному порядку, генератор `WHERE` навчали з підкріпленням і оцінювали не дослівний збіг з еталоном, а результат виконання запиту. SQLNet [13], у свою чергу, розв'язувала ту саму проблему без навчання з підкріпленням. Вона заповнювала складові фіксованого шаблону SQL і передбачала колонки умов як невпорядковану множину із механізмом уваги до колонок.

Попередньо навчені мовні моделі. Наприкінці 2010-х років у Text-to-SQL почали використовувати попередньо навчені трансформерні моделі. Ключовою відмінністю від попереднього етапу стало попереднє навчання моделі на великих текстових корпусах із подальшою адаптацією до Text-to-SQL. Під час адаптації модель донавчали на прикладах, що пов'язували питання і схему бази даних з еталонним SQL-запитом. RAT-SQL кодувала зв'язки між елементами схеми та відповідності між словами питання й елементами схеми за допомогою механізму самоуваги з урахуванням відношень. У варіанті RAT-SQL + BERT початкові представлення слів питання й назв елементів схеми отримували за допомогою BERT [14]. TaBERT створили шляхом продовження попереднього навчання BERT на таблицях і пов'язаних із ними текстах. Потім TaBERT використовували як кодувальник у семантичному парсері та донавчали разом із ним на прикладах Text-to-SQL [15]. У PICARD під час декодування донавченої T5 інкрементальний синтаксичний аналіз відхиляв токени, які не могли бути допустимим продовженням SQL-запиту [16].

Великі мовні моделі. Перехід до великих мовних моделей був пов'язаний із масштабуванням попередньо навчених моделей за кількістю параметрів, обсягом навчальних даних і обчислювальними витратами. Довше контекстне вікно давало змогу одночасно подавати моделі інструкцію, опис схеми бази даних та, за потреби, приклади «питання — SQL». У 2022 році Rajkumar та ін. показали можливість використання Codex для генерації SQL у такій постановці без спеціалізованого донавчання [17]. DIN-

SQL розкладає задачу на зв'язування зі схемою, декомпозицію, генерацію SQL і самокорекцію [18]. DAIL-SQL досліджує добір і подання прикладів для навчання в контексті [19]. Водночас донавчання зберегло своє значення. Великі мовні моделі донавчають повністю або оновлюють лише невелику частину параметрів, наприклад низькорангові адаптери LoRA [7, 20]. Зокрема, моделі OmniSQL донавчено на синтетичному наборі SynSQL-2.5M разом із тренувальними вибірками Spider і BIRD, а цільова відповідь містить покрокове пояснення та фінальний SQL-запит [21].

1.2 Набори даних і метрики оцінювання систем Text-to-SQL

Набори даних Text-to-SQL містять приклади, на яких навчають і оцінюють моделі. Результат оцінювання залежить від складу набору, способу його розбиття на вибірки та метрик, за якими зіставляють згенерований SQL-запит з еталонним.

1.2.1 Набори даних Text-to-SQL

Набори даних Text-to-SQL відрізняються роллю в експерименті та способом формування. Еталонні набори задають фіксований протокол оцінювання, синтетичні корпуси збільшують обсяг навчальних прикладів, а діагностичні набори перевіряють стійкість моделей до контрольованих змін у питанні, базі даних або SQL-запиті. Основні характеристики розглянутих наборів узагальнено в табл. 1.1.

Таблиця 1.1 – Основні характеристики наборів даних Text-to-SQL

Набір	Формування	К-сть БД	К-сть прикладів	Характеристики
WikiSQL [12]	шаблон, краудсорсинг	24 241	80 654	міждоменний, однотабличний
Spider [4]	ручне анотування	200	10 181	міждоменний, багатотабличний
BIRD [5]	ручне анотування	95	12 751	міждоменний, великі БД
Gretel [22]	мовна модель	—	105 851	міждоменний
SynSQL-2.5M [21]	мовна модель	16 583	2 544 390	міждоменний
Dr.Spider [23]	на основі Spider		15 269	діагностичний

Окремий екземпляр корпусу подамо як кортеж

$$x_i = (q_i, S_i, D_i, E_i, y_i^*), \tag{1.3}$$

де позначення відповідають виразам (1.1) і (1.2), а y_i^* — еталонний SQL-запит. Корпус $\mathcal{C} = \{x_i\}_{i=1}^{|\mathcal{C}|}$ є скінченною множиною таких екземплярів. Його розбивають на тренувальну, валідаційну та тестову вибірки, що попарно не перетинаються:

$$\mathcal{C} = \mathcal{C}_{\text{train}} \cup \mathcal{C}_{\text{dev}} \cup \mathcal{C}_{\text{test}}. \tag{1.4}$$

Через $\mathcal{B}(\mathcal{C}') = \{(S_i, D_i) : x_i \in \mathcal{C}'\}$ позначимо множину баз даних, до яких звертаються приклади вибірки $\mathcal{C}' \subseteq \mathcal{C}$. Якщо, крім того,

$$\mathcal{B}(\mathcal{C}_a) \cap \mathcal{B}(\mathcal{C}_b) = \varnothing, \qquad a \neq b, \quad a, b \in \{\text{train}, \text{dev}, \text{test}\}, \tag{1.5}$$

то жодна база даних не повторюється в різних вибірках, і тестові приклади звертаються лише до баз, яких не було в тренувальній вибірці.

Еволюція еталонних наборів. Розвиток методів Text-to-SQL супроводжувався зростанням вимог до еталонних наборів. Їхнє охоплення розширилося від однотабличних запитів до багатотабличних схем, а згодом — до великих баз даних, у яких важливими є не лише структура, а й збережені значення. Першим великим міждоменним набором став WikiSQL, що містить 80 654 приклади на 24 241 таблиці Вікіпедії [7, 12]. SQL-запит і початкове формулювання питання генерували за шаблоном, після чого формулювання перефразовували засобами краудсорсингу. Кожний приклад пов'язаний з однією таблицею. Запити охоплюють вибір колонки, агрегування та умови `WHERE`, але не з'єднання таблиць, вкладені запити, групування чи операції над множинами. Авторський поділ містить 56 355 тренувальних, 8 421 валідаційний і 15 878 тестових прикладів. Таблиці між цими вибірками не перетинаються, тобто умова (1.5) виконується для баз, що складаються з однієї таблиці. Водночас різні таблиці можуть мати однакові схеми. Близько 70 % тестових пар «питання — таблиця» мають схему, що трапляється в тренувальній вибірці [24]. Тому WikiSQL лише частково перевіряє узагальнення на нові схеми і не перевіряє здатності працювати зі складною реляційною схемою [12, 25].

Обмеження однотабличної постановки усуває набір Spider [4]. На мо-

мент публікації він був єдиним набором Text-to-SQL, що поєднував багатотабличні бази даних різних предметних областей зі складними SQL-запитами. Структуру набору, зокрема розбиття за базами даних, показано на рис. 1.3.

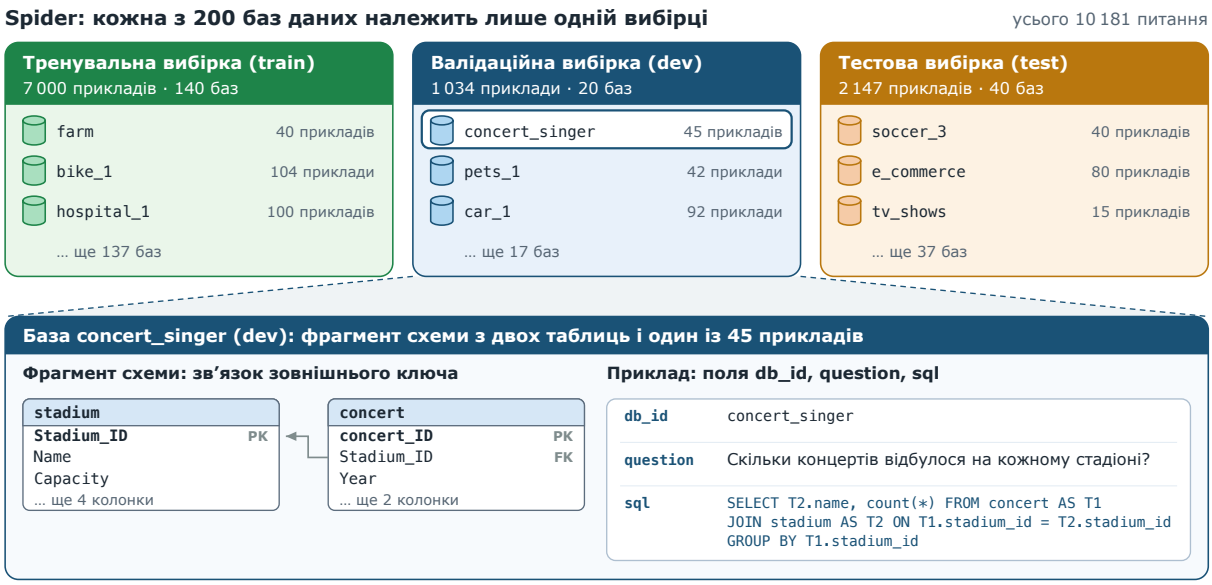


Рисунок 1.3 – Структура набору Spider: вибірки, бази даних і приклади

Пари «питання — SQL» сформовано шляхом ручного анотування для 200 баз даних зі 138 предметних областей. Основний корпус містить 10 181 питання та 5 693 унікальні SQL-запити. На відміну від WikiSQL, схеми містять кілька пов'язаних таблиць із первинними та зовнішніми ключами, а запити охоплюють з'єднання, групування, вкладені запити й операції над множинами. Автори також поділяють запити на прості, середні, складні та особливо складні. У наукових публікаціях Spider залишається найпоширенішим еталонним набором для оцінювання методів Text-to-SQL [1, 26].

Як і у WikiSQL, бази даних Spider не повторюються між вибірками, тобто виконується умова (1.5), але вже для багатотабличних схем. Тому оцінювання на Spider перевіряє узагальнення побудови SQL на нові реляційні схеми та предметні області.

Водночас автори BIRD зазначають, що WikiSQL і Spider зосереджені передусім на схемі бази даних, а не на її значеннях. У середньому на одну базу припадає 17 рядків у WikiSQL і близько 2 тис. рядків у Spider [5].

Набір BIRD доповнює Spider великими базами з реальним наповненням [5]. Він містить 12 751 вручну анотовану пару «питання — SQL» для 95 баз даних із 37 професійних предметних областей. Бази розподілено між тре-

нувальною, валідаційною та тестовою вибірками без перетину (69, 11 і 15 баз), тобто умова (1.5) виконується і для BIRD. Сумарний обсяг баз становить близько 33,4 ГБ, а на одну базу в середньому припадає близько 549 тис. рядків. Значення в базах можуть містити помилки та шум, а для інтерпретації питання можуть бути потрібні конкретні значення, синоніми, правила числового обчислення або предметні знання. На відміну від попередніх наборів, BIRD доповнює приклади окремим полем `evidence`, у якому подано такі відомості. Це поле відповідає компоненті E_i у виразі (1.3). Доступ моделі до цього поля є частиною умов експерименту: у термінах виразу (1.1) модель отримує або (q_i, S_i, E_i), або $(q_i, S_i, \varnothing)$.

Синтетичні навчальні корпуси. Ручне анотування обмежує обсяг Spider і BIRD приблизно десятком тисяч прикладів. Синтетичні корпуси, згенеровані мовними моделями, збільшують обсяг навчальних даних на один–два порядки [7, 21]. Відкритий корпус Gretel `synthetic_text_to_sql`, створений за допомогою Gretel Navigator, містить 105 851 приклад зі 100 предметних областей [22]. Корпус SynSQL-2.5M містить 2 544 390 прикладів на 16 583 синтетичних базах SQLite і використаний для донавчання моделей OmniSQL [21]. Водночас під час експертної перевірки 1 000 випадкових прикладів SynSQL-2.5M повністю правильними визнано 86 % [21]. Отже, синтетичні корпуси доповнюють навчальні дані, але не замінюють еталонних наборів із фіксованим протоколом оцінювання.

Діагностичні набори. Окремий випадок становлять набори, призначені для перевірки стійкості моделей до контрольованих змін вхідних даних. Розбиття (1.5) дає змогу оцінити узагальнення на нові бази даних, але не показує, як результат змінюється після перефразування питання, перейменування елементів схеми або модифікації значень у базі. Для такої перевірки Chang та ін. побудували на основі валідаційної вибірки Spider діагностичний набір Dr.Spider [23]. Він містить 17 типів контрольованих модифікацій: 3 для бази даних, 9 для питання і 5 для SQL-запиту, разом 15 269 модифікованих прикладів, зіставлених із вихідними. На Dr.Spider найстійкіша з перевірених систем, PICARD, втрачає в середньому 14,0 % відносної макроусередненої точності виконання, а для модифікацій із заміною значень у базі на семантично еквівалентні втрата сягає 50,7 %. Отже, Dr.Spider показує, що моделі Text-to-SQL можуть бути чутливими до семантично контрольованих змін питання, схеми

або даних, навіть якщо зміст задачі зберігається.

1.2.2 Метрики оцінювання моделей Text-to-SQL

Метрики Text-to-SQL перевіряють різні властивості згенерованого запиту. Структурні метрики зіставляють його компоненти з еталонним SQL, тоді як метрики виконання порівнюють результати або час роботи запитів. Нехай оцінювана вибірка містить N прикладів. Для i-го прикладу позначимо еталонний запит через Y_i, згенерований запит через $\hat{Y}_i$, а результати їх виконання через V_i та $\hat{V}_i$. Індикаторна функція $\mathbf{1}(\cdot)$ дорівнює одиниці, якщо умова виконується, і нулю в іншому випадку.

Зіставлення компонент. Component matching (CM) зіставляє структурні компоненти еталонного та згенерованого SQL-запитів. До них належать конструкції `SELECT`, `WHERE`, `GROUP BY`, `ORDER BY`, а також оператори й модифікатори запиту [4]. Нехай C позначає одну таку компоненту, а Y_i^C і $\hat{Y}_i^C$ — її нормалізоване подання в еталонному та згенерованому запитах. Точність збігу компоненти визначають як частку прикладів, у яких ці подання збігаються [7]

$$\mathrm{CM}_C = \frac{1}{N}\sum_{i=1}^{N} \mathbf{1}\left(Y_i^C = \hat{Y}_i^C\right). \tag{1.6}$$

Правила нормалізації залежать від компоненти. Для умов і групування порядок складників зазвичай не враховують, тоді як для `ORDER BY` істотними є послідовність виразів і напрямок сортування. Частковий збіг усередині компоненти не дає часткового бала. На практиці для кожної компоненти також звітують точність, повноту та F_1 [4]. Такий аналіз показує, у якій частині SQL виникла помилка, але не дає єдиного рішення про правильність усього запиту.

Повний структурний збіг. Exact match (EM) перевіряє повний структурний збіг еталонного та згенерованого SQL-запитів після їх розбиття на компоненти [4, 7]. Нехай $\mathcal{C}$ позначає набір компонентів SQL, а Y_i^C і $\hat{Y}_i^C$ — компоненту C в еталонному та згенерованому запитах. EM визначають як частку прикладів, у яких збігаються всі компоненти

$$\mathrm{EM} = \frac{1}{N}\sum_{i=1}^{N} \mathbf{1}\left(\bigwedge_{C\in\mathcal{C}} Y_i^C = \hat{Y}_i^C\right). \tag{1.7}$$

Перед структурним порівнянням оцінювач може нормалізувати літеральні значення, ознаку `DISTINCT` та еквівалентні колонки. Тому EM залежить не лише від згенерованих запитів, а й від правил і налаштувань оцінювача.

Точність виконання. Execution accuracy (EX) перевіряє, чи повертають згенерований та еталонний запити однаковий результат на тому самому стані бази [7]

$$\text{EX} = \frac{1}{N}\sum_{i=1}^{N}\mathbf{1}(V_i = \hat{V}_i). \tag{1.8}$$

Помилка виконання згенерованого запиту означає невдалий приклад. Під час обчислення EX потрібно зафіксувати, чи враховує оцінювач порядок колонок і рядків, а також повторювані значення, оскільки різні реалізації можуть застосовувати різні правила зіставлення результатів. EX допускає структурно відмінний SQL, якщо він повертає той самий результат на поточному стані бази. Такий збіг не доводить семантичної еквівалентності, оскільки два різні запити можуть випадково повернути однакову відповідь, зокрема порожню [27].

Точність на тестовому наборі станів. Test suite accuracy (TS) зменшує залежність від одного стану бази. Для кожного прикладу випадково створені стани бази дистилюють у множину $\mathcal{D}_i$, яка розрізняє якомога більше близьких неправильних варіантів еталонного запиту. Запит зараховують лише тоді, коли його результат збігається з еталонним на кожному стані з $\mathcal{D}_i$ [28]

$$\text{TS} = \frac{1}{N}\sum_{i=1}^{N}\mathbf{1}\left(\forall D \in \mathcal{D}_i \quad \text{Exec}(\hat{Y}_i, D) = \text{Exec}(Y_i, D)\right). \tag{1.9}$$

Якщо запити збігаються на всіх відібраних станах, випадкова еквівалентність менш імовірна, ніж за EX на одній базі. Проте скінченний тестовий набір не доводить рівність результатів для кожного можливого наповнення схеми.

Показник правильної ефективності. Valid efficiency score (VES), запропонований у BIRD, ураховує правильність результату й відносний час виконання. Нехай $E(Y_i)$ та $E(\hat{Y}_i)$ позначають оцінки часу виконання еталонного і згенерованого запитів. Формулу VES та відносну ефективність R визнача-

ють так [5]

$$\text{VES} = \frac{1}{N}\sum_{i=1}^{N} \mathbf{1}(V_i = \hat{V}_i) \cdot R(Y_i, \hat{Y}_i), \qquad R(Y_i, \hat{Y}_i) = \sqrt{\frac{E(Y_i)}{E(\hat{Y}_i)}}. \tag{1.10}$$

До VES потрапляють лише запити, що повертають правильний результат, а неправильний за результатом запит має нульовий внесок незалежно від швидкості. Для правильних запитів множник R відображає відносний час виконання і дорівнює одиниці за однакового часу, перевищує одиницю для швидшого згенерованого запиту та є меншим за одиницю для повільнішого. Метрика оцінює виміряний час, а не синтаксичну складність запиту, тому з'єднання чи підзапити впливають на бал лише через результат і час виконання. Для відтворюваного порівняння час вимірюють в однаковому апаратному середовищі за багаторазового виконання, з вилученням часових викидів та усередненням решти вимірювань [5].

В академічних дослідженнях Text-to-SQL найчастіше використовують EM та EX. CM і TS уточнюють відповідно місце структурної помилки та стійкість результату на кількох станах бази. У комерційних інформаційних системах часто істотною є швидкодія, тому VES може бути додатковою метрикою або орієнтиром для перевірки того, чи модель генерує ефективний SQL.

Інтерпретація значень наведених метрик спирається на коректність еталонного запиту Y_i. Якщо він помилковий, збіг із ним не обов'язково свідчить про правильність згенерованого запиту, а розбіжність не обов'язково означає помилку моделі. Тому дефекти валідаційних і тестових анотацій можуть спотворювати порівняння моделей.

1.3 Методи адаптації великих мовних моделей до задачі Text-to-SQL

До основних підходів адаптації великих мовних моделей до задачі Text-to-SQL належать навчання в контексті та донавчання з учителем, зокрема його варіанти LoRA і QLoRA [1, 19, 29].

1.3.1 Навчання в контексті

Навчання в контексті (англ. *in-context learning*, ICL) дає змогу адаптувати велику мовну модель до задачі Text-to-SQL без її додаткового донавчання. Для цього модель отримує текстовий запит, що містить інструкцію, опис схеми

бази даних, питання користувача та, за потреби, приклади очікуваного перетворення природномовного запиту в SQL. Такі приклади використовуються лише під час обробки поточного запиту й слугують орієнтиром для побудови відповіді [29].

Нехай π_θ — мовна модель з параметрами θ, а I — інструкція до задачі Text-to-SQL. Повний вхід моделі позначимо через P і називатимемо запитом до моделі (англ. *prompt*). Символ $\oplus$ означає структуроване об'єднання складників цього запиту. Для екземпляра x_i запит без додаткових прикладів має вигляд

$$P_i^{(0)} = I \oplus S_i \oplus E_i \oplus q_i. \tag{1.11}$$

Тут q_i, S_i та E_i позначають відповідно питання, схему цільової бази даних і додаткові відомості для прикладу x_i, визначеного у виразі (1.3). Якщо додаткових відомостей немає, компоненту $E_i = \varnothing$ опускають. Схему S_i попередньо подають текстом. Таке подання називають серіалізацією схеми. Воно може містити таблиці, колонки, типи даних, первинні та зовнішні ключі.

Використання лише $P_i^{(0)}$ відповідає режиму без прикладів у запиті (zero-shot). У режимі з кількома прикладами у запиті (few-shot prompting) до цільового входу додають приклади очікуваного перетворення. Приклад у запиті

$$d_j = S_j \oplus E_j \oplus q_j \oplus y_j^*$$

містить вхідні дані й еталонний SQL-запит. Для прикладів $d_{j_1}, \ldots, d_{j_k}$ повний запит записують як

$$P_i^{(k)} = d_{j_1} \oplus \cdots \oplus d_{j_k} \oplus P_i^{(0)}, \qquad \widehat{y}_i = \pi_\theta\big(P_i^{(k)}\big). \tag{1.12}$$

Огляд Hong та ін. відносить до ICL як режим без прикладів у запиті, так і режим із кількома прикладами у запиті [29]. У виразі (1.12) ці випадки відповідають $k = 0$ та $k > 0$. Приклади для запиту добирають із доступної тренувальної вибірки без використання еталонного запиту цільового екземпляра.

Якість ICL залежить від добору та подання прикладів у запиті. Істотними є подібність питань, очікувана структура SQL, складність прикладів і їхня предметна область. Значення також мають формат і порядок складників запиту. DAIL-SQL систематично досліджує представлення питання, добір та

організацію прикладів у запиті [19]. За даними огляду Hong та ін., приклади з тієї самої предметної області зазвичай ефективніші за приклади з інших областей [29].

Кількість прикладів у запиті обмежує контекстне вікно моделі. У Text-to-SQL значну його частину може займати серіалізована схема, особливо для баз із великою кількістю таблиць. До контексту також можуть входити значення з бази й додаткові відомості E_i. Тому збільшення k скорочує обсяг контексту, доступний для опису цільової бази.

ICL не потребує окремого циклу донавчання й придатне для моделей, параметри яких недоступні користувачеві. Водночас довший запит збільшує обчислювальні витрати під час кожного виведення. Результат також чутливий до формулювання інструкції та складу прикладів у запиті. Через невелику кількість демонстрацій у запиті кожен помилковий приклад може істотно вплинути на структуру та семантику згенерованого SQL. Декомпозиція задачі, покрокове міркування та уточнення за результатом виконання розширюють базову схему ICL [29]. Ці прийоми організовують процес виведення, тому далі окремо не розглядаються.

1.3.2 Донавчання з учителем

Донавчання з учителем (англ. *supervised fine-tuning*, SFT) — це адаптація попередньо навченої авторегресійної моделі на тренувальних парах «контекст — цільова послідовність» [29]. У задачі Text-to-SQL для кожного екземпляра $x_i \in \mathcal{C}_{\text{train}}$ навчальну пару утворюють запит $P_i^{(0)}$, визначений у (1.11), та еталонний SQL-запит $y_i^* = (y_{i,1}^*, \ldots, y_{i,T_i}^*)$ із T_i токенів.

Імовірність еталонного запиту має авторегресійний розклад

$$p_\theta(y_i^* \mid P_i^{(0)}) = \prod_{t=1}^{T_i} p_\theta\big(y_{i,t}^* \mid P_i^{(0)}, y_{i,<t}^*\big). \tag{1.13}$$

Кожний множник задає ймовірність наступного токена за вхідним запитом і попередніми еталонними токенами. Якщо втрату обчислюють лише для токенів SQL, функція втрат SFT має вигляд

$$\mathcal{L}_{\text{SFT}}(\theta) = -\sum_{x_i \in \mathcal{C}_{\text{train}}} \sum_{t=1}^{T_i} \log p_\theta\big(y_{i,t}^* \mid P_i^{(0)}, y_{i,<t}^*\big). \tag{1.14}$$

Сума (1.14) є токенною крос-ентропією еталонного SQL, а її мінімізація збільшує ймовірність цієї послідовності за заданим входом. DAIL-SQL реалізує цю схему, маскуючи токени запиту під час обчислення втрат [19]. У CodeS після інкрементального попереднього навчання модель повністю донавчають для генерації SQL за описом бази даних і питанням [30].

SFT визначає навчальну ціль, але саме по собі не визначає множину оновлюваних параметрів. За повного донавчання (англ. *full fine-tuning*) оптимізацію виконують за всіма параметрами моделі [20]:

$$\theta^* \in \arg\min_{\theta} \mathcal{L}_{\text{SFT}}(\theta). \tag{1.15}$$

Повне донавчання потребує зберігання градієнтів і станів оптимізатора для всіх оновлюваних параметрів. Загальні витрати пам'яті під час навчання також охоплюють ваги моделі й активації, обсяг яких залежить від довжини послідовності та кількості прикладів, які модель опрацьовує одночасно [2]. Для моделей із мільярдами параметрів такий режим створює значні вимоги до пам'яті.

1.3.3 Донавчання за допомогою LoRA і QLoRA

Ефективне за параметрами донавчання (англ. *parameter-efficient fine-tuning*, PEFT) скорочує кількість оновлюваних параметрів [2]. У методах на зразок LoRA основні параметри попередньо навченої моделі θ_0 фіксують, а оптимізують меншу множину параметрів ϕ. Для моделі $\pi_{\theta_0,\phi}$ цей випадок можна записати як

$$\phi^* \in \arg\min_{\phi} \mathcal{L}_{\text{SFT}}(\theta_0, \phi), \qquad \theta_0 \text{ фіксовано}. \tag{1.16}$$

Отже, PEFT змінює множину параметрів, які оптимізують, але не навчальну ціль SFT.

Адаптація низького рангу (англ. *low-rank adaptation*, LoRA) задає параметри ϕ двома матрицями низького рангу [20]. Нехай $W_0 \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ — фіксована матриця лінійної проєкції. Для входу x вихід має вигляд

$$h = W_0 x + \frac{\alpha}{r} BAx, \tag{1.17}$$

де $A \in \mathbb{R}^{r \times d_{\text{in}}}$, $B \in \mathbb{R}^{d_{\text{out}} \times r}$, а $r \ll \min(d_{\text{in}}, d_{\text{out}})$. Добуток BA є низькоранговим

оновленням матриці ваг, а коефіцієнт α/r масштабує відповідний доданок у (1.17). Матрицю A ініціалізують випадково, а B — нулями. Під час донавчання W_0 лишається фіксованою, а оптимізатор оновлює лише A і B. Після навчання матриці A і B більше не змінюються, тому їхній добуток можна один раз додати до W_0 [20]:

$$W = W_0 + \frac{\alpha}{r} BA. \tag{1.18}$$

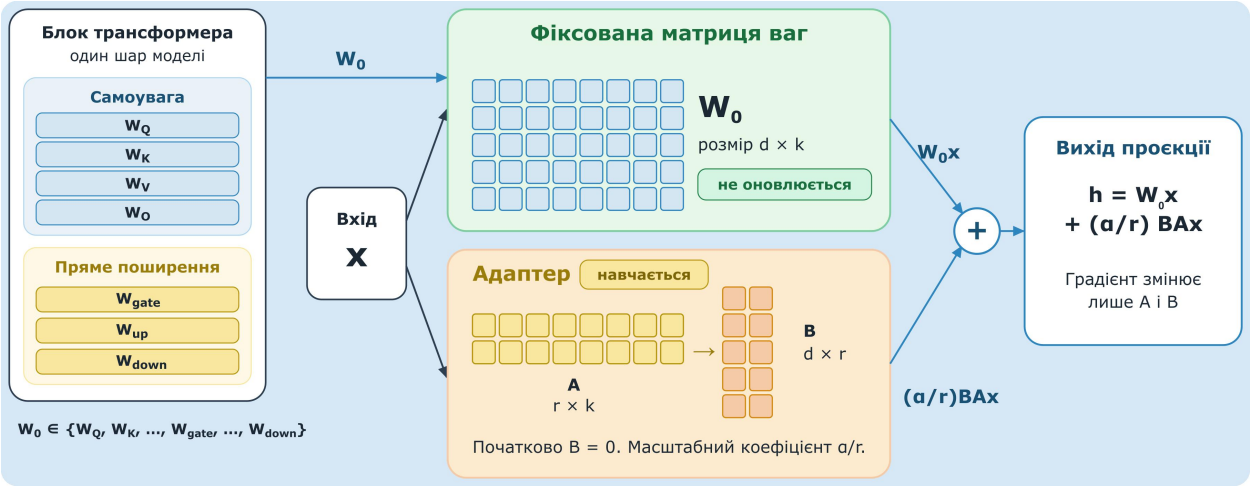


Рисунок 1.4 – Низькорангове оновлення лінійної проєкції. Матриця W_0 фіксована, а A і B навчаються. Початково $B = 0$.

Hu та ін. в основних експериментах адаптують проєкції уваги W_Q і W_V, низькорангову адаптацію яких показано на рис. 1.4 [20]. Подальші варіанти поширюють низькорангові адаптери на інші проєкції уваги та блок прямого поширення [2]. У подальшому експерименті з моделлю зі шлюзованим блоком прямого поширення адаптери встановлюються на W_Q, W_K, W_V, W_O, W_{gate}, W_{up} і W_{down}.

QLoRA зменшує витрати пам'яті, поєднуючи LoRA з 4-бітним квантуванням основної моделі [31]. Фіксовані ваги зберігають у форматі NormalFloat 4 (NF4). Під час множення їх деквантують до 16-бітного обчислювального типу, а градієнти обчислюють лише для адаптерів. Ваги квантують блоками, і кожен блок має власний масштабний коефіцієнт. Подвійне квантування (англ. *double quantization*) зберігає ці коефіцієнти у 8-бітному форматі замість 32-бітного. Це зменшує додаткові витрати пам'яті приблизно з 0,5 до 0,127 біта на параметр [31]. Сторінкові оптимізатори (англ. *paged optimizers*) зменшують пікові витрати пам'яті під час навчання.

На WikiSQL LoRA досягла точності повного донавчання GPT-3 175B у межах похибки експерименту [20]. QLoRA з форматом NF4 не поступилася 16-бітній LoRA за середньою точністю на MMLU [31]. Водночас на складніших задачах LoRA з поширеними низькими рангами може відставати від повного донавчання [2, 32].

1.3.4 Вплив якості навчальних даних на результати донавчання

Повне донавчання, LoRA і QLoRA оптимізують ту саму функцію втрат SFT (1.14). Якщо еталонний SQL-запит є помилковим, модель отримує сигнал збільшувати ймовірність помилкової послідовності. Отже, якість навчальних анотацій впливає на донавчання незалежно від того, скільки параметрів оновлюють.

Емпіричні результати показують, що для донавчання важливий не лише обсяг корпусу, а й добір прикладів. Dettmers та ін. показали, що модель, донавчена за допомогою QLoRA на 9 тис. діалогових прикладів OASST1, перевершує в діалогових задачах модель, донавчену на 450 тис. прикладів FLAN v2 [31]. У Text-to-SQL додавання синтетичного корпусу нижчої якості Gretel-Synth знижує точність моделі, хоча обсяг тренувальної вибірки зростає [33]. У цих роботах змінюється склад вибірки, тому вони підтверджують загальний вплив якості даних, але не ізолюють ефект виправлення тих самих анотацій.

Безпосередній зв'язок між виправленням навчальних анотацій і результатами донавчання показано в SQLDriller: автори виправляють еталонні SQL-запити в наявних тренувальних парах, не змінюючи складу вибірки [34]. Після цього автори донавчили RESDSQL, Graphix-T5 і CodeS та порівняли результати з навчанням на початкових анотаціях. Використання виправлених пар підвищило точність на тестовому наборі станів. Водночас окремі типи виправлень у цих експериментах не оцінювали, а LoRA і QLoRA не розглядали.

Подальші роботи підтверджують вплив виправлень і для параметрично ефективної адаптації. У DCMM-SQL автоматично замінено 143 еталонні запити BIRD і 95 запитів Spider [35]. Після SFT з LoRA точність виконання моделей Qwen2.5-Coder зросла на 1,76 і 1,30 відсоткового пункту для моделей 7B і 32B відповідно. Zhu і Kang вручну виправили 372 із 600 випадково

відібраних тренувальних пар BIRD та показали, що навчання з підкріпленням із LoRA на початкових анотаціях дає на 5,7–12,1 % нижчу точність, ніж на виправлених [36].

1.4 Існуючі методи аудиту наборів даних Text-to-SQL

Як показано в підрозділах 1.2.2 і 1.3.4, дефекти наборів даних Text-to-SQL спотворюють оцінювання моделей і навчальний сигнал під час донавчання. Тому потрібен аудит набору, тобто систематична перевірка, яка встановлює наявність, місце й тип дефектів. Далі розглянуто наявні методи аудиту наборів даних.

1.4.1 Ручний аудит і виправлення анотацій

Формування сучасних наборів даних Text-to-SQL зазвичай передбачає окремий етап перевірки анотацій. Під час підготовки набору даних Spider питання природною мовою та відповідні SQL-запити для кожної бази даних укладали вісім студентів комп'ютерних наук із високим рівнем компетентності в SQL. Окремими етапами були рецензування SQL-запитів, перегляд і перефразування питань та остаточне опрацювання, по 150 людино-годин кожний [4]. У BIRD питання і SQL-запити складали різні учасники, а запити проходили подвійне сліпе анотування та експертну перевірку [5]. Такі процедури зменшують кількість помилок, але не усувають їх повністю. Подальші ручні перевірки Spider і BIRD показали, що помилки зберігаються навіть після багатоетапного рецензування.

Pourreza і Rafiei вручну переоцінили 102 пари Spider Dev, на яких за метрикою точності виконання одночасно помилилися DIN-SQL і T5-large+PICARD [37]. Експерти визнали правильними 67,3 % еталонних запитів, тобто 32,7 % запитів у цій вибірці були неправильними або не відповідали прийнятому тлумаченню питання. Цей результат не можна переносити на всі 1 034 пари Spider Dev, оскільки 102 пари відібрано саме за спільною помилкою двох систем. Окремо, щоб під час оцінювання коректно врахувати рівні значення, автори переписали 206 із 1 034 запитів Spider Dev.

Wretblad та ін. дослідили шум у вибраних доменах BIRD [38]. У повному домені `financial` дефект містять 52 із 106 прикладів (49 %). У випадкових вибірках по 20 прикладів частка становить 9 із 20 (45 %) для `california_schools` і 3 із 20 (15 %) для `superhero`. Помилки стосуються

як формулювання питань, так і еталонного SQL. Окремо частка помилкових еталонних запитів у `financial` становить 22 із 106 (20,7 %). Автори переоцінили моделі після виправлення еталонного SQL і після додаткового виправлення питань. Оскільки перевірено окремі домени, а дві з наведених часток отримано на вибірках по 20 прикладів, ці числа не є оцінкою всього BIRD.

Для оцінювання SQLDriller автори вручну перевірили по 500 випадкових пар із Spider Train і BIRD Train та виявили помилки відповідно у 183 (36,6 %) і 272 (54,4 %) парах [34]. Отриману розмітку використали як еталон для оцінювання автоматичного виявлення та виправлення помилок. Окремо за допомогою SQLDriller автоматично сформували виправлення для Spider Test і BIRD Dev, після чого чотири студенти старших курсів із підготовкою в SQL вручну переглянули всі пари, витративши по 80 людино-годин на набір. У початкових анотаціях помилковими були 810 із 2 147 пар Spider Test (37,7 %) і 768 із 1 534 пар BIRD Dev (50,1 %).

Наведені результати показують, що помилки зберігаються навіть у наборах даних, які пройшли багатоетапну перевірку за участю фахівців. Ручний аудит дає змогу врахувати предметну семантику, неоднозначність питань та альтернативні правильні SQL-запити, проте потребує значних витрат праці. Для синтетичних корпусів, обсяг яких перевищує 100 тис. і може сягати кількох мільйонів прикладів, суцільна ручна перевірка стає практично нездійсненною. Це зумовлює потребу в автоматизованих і відтворюваних засобах аудиту.

1.4.2 Профілювання наборів, перевірка бази даних і виконання еталонних запитів

Профілювання наборів даних. Найближчим до багатовимірного аудиту за формою результату є профілювання наборів Text-to-SQL за детермінованими показниками. Mitsopoulou і Koutrika характеризують набори дванадцятьма показниками, згрупованими за SQL-запитами, базами даних і питаннями, і пояснюють ними погіршення перенесення моделі між наборами [26]. Такі показники дають контекст для знахідок аудиту. Зокрема, вони показують, чи не зводиться набір до простих запитів, і дають змогу зіставляти частку дефектів зі складністю запитів і схем. Проте профіль не перевіряє коректності

складових набору, тому не встановлює, які питання, еталонні запити чи бази даних містять дефекти.

База даних. Профілювання реляційних даних виявляє залежності включення, унікальні комбінації колонок і кандидатів у зовнішні ключі [39], а SchemaCrawler забезпечує статичну перевірку проєктування схеми [40]. Memari та ін. аналізують порушення зовнішніх ключів за семантики `NULL`, коли примусовий контроль ключів вимкнено [41], що відповідає базам SQLite [42]. Такі перевірки доступні у вбудованих засобах СКБД і в окремих інструментах, проте їхній результат стосується бази загалом і не пов'язаний з питаннями та еталонними запитами набору. Для Text-to-SQL порушення посилальної цілісності призводить до втрати рядків під час з'єднання таблиць, тому результат еталонного запиту при обчисленні EX і TS залежить від дефекту бази, а не лише від питання. Неоголошені або суперечливі зовнішні ключі у схемі S_i спотворюють також опис зв'язків таблиць у запиті до моделі (1.11). Такі дефекти відзначено і в дослідженнях наборів. Shen та ін. побічно, під час аналізу помилок згенерованих запитів, виявили численні порушення посилальної цілісності в базі flight_2 набору Spider і вилучили цю базу з експериментів [43].

Виконання запиту. Розбір і виконання еталонного запиту на відповідній базі виявляють синтаксичні помилки, помилки часу виконання та порожні результати. Приклади з порожнім еталонним результатом виключають з оцінювання за виконанням або вважають помилковими [26, 28]. Крім того, результат еталонного запиту може бути недетермінованим, тобто залежати від порядку виконання в СКБД. За оцінкою Pourreza і Rafiei, 20–23 % запитів dev-вибірок Spider і BIRD можуть повертати лише частину рівних за критерієм упорядкування рядків, тому правильний запит може не збігтися з еталоном, а автори переписали лише ту частину таких запитів, яку можна змінити механічно [37]. Концептуальна схема валідації Pandey та ін. [44] охоплює перевірку побудови запиту, цілісності даних і зворотного зв'язку, але орієнтована на супровід окремих запитів у промисловій системі й не має відкритої реалізації.

Отже, дефекти бази даних і недетерміновані результати еталонних запитів визнано джерелами помилок оцінювання, проте наявні роботи усувають їх вибірково, вилученням бази або переписуванням запитів. Вилучення бази усуває всі її пари, хоча дефект зачіпає лише ті, чий еталонний запит спи-

рається на порушений зв'язок. Окремі засоби формують висновки про різні об'єкти, а саме базу, запит або пару, які неможливо зіставити між собою. Тому цілісність бази й виконання еталонного запиту мають бути окремими вимірами спільного профілю кожної анотації, за яким можна встановити джерело дефекту й спосіб його виправлення.

1.4.3 Виявлення антипатернів SQL

Синтаксична правильність і успішне виконання ще не гарантують коректності SQL-запиту. Конструкції, які СКБД виконує без помилки, але які призводять до неправильних або неоднозначних результатів чи ускладнюють супровід бази даних, називають антипатернами SQL. Класичну систематизацію таких конструкцій запропонував Karwin [45]. Його каталог охоплює як логічне проєктування схем, так і побудову запитів, зокрема неоднозначне групування, неявний перелік колонок і некоректне використання значення `NULL`.

На основі каталогу Karwin і усталеної практики розроблення баз даних реалізовано відкритий інструмент sqlcheck [46] із 17 правилами перевірки запитів. Більшість цих правил спрямовано на конструкції, що сповільнюють виконання запиту, а не на ті, що спричиняють хибний чи недетермінований результат. Реалізація має істотні обмеження. Інструмент шукає в тексті запитів підрядки за регулярними виразами без синтаксичного розбору, тому правило спрацьовує на ключове слово, а не на дефект. Наприклад, правило про неагреговані колонки спрацьовує на будь-який запит із `GROUP BY`, зокрема на коректний, тоді як запит з агрегатом і неагрегованою колонкою без `GROUP BY` залишається без знахідки. Крім того, інструмент не використовує схему бази даних, хоча саме її обмеження відрізняють дефект від безпечної конструкції. Наприклад, `NOT IN` з підзапитом небезпечний, лише коли колонка, яку повертає підзапит, може містити `NULL`. До того ж інструмент орієнтовано на прикладні системи, а не на набори даних Text-to-SQL.

У пізнішій роботі [47] на основі цього інструменту запропоновано підхід SQLCheck до виявлення, ранжування й виправлення антипатернів у коді прикладних систем баз даних. Його каталог містить 26 класів антипатернів, згрупованих у чотири категорії, а саме логічне проєктування схеми, фізичне проєктування, побудова запитів і властивості даних. Для синтаксичного розбору запиту використано аналізатор sqlparse, а правила виявлення врахову-

ють контекст прикладної системи. Контекст містить схему, інші запити, статистики й вибірки даних і має зменшувати кількість хибних спрацювань і пропусків. Знахідки ранжують за зваженою сумою показників впливу з вагами, які задає розробник.

Вихідний код цих розширень не опубліковано, тому підхід доступний лише як методологія, яка для аудиту еталонних запитів наборів Text-to-SQL має три обмеження. По-перше, лише сім із 26 класів каталогу стосуються побудови запитів, і, за оцінкою авторів, на точність результату з них впливають два. По-друге, аналізатор sqlparse не будує повного абстрактного синтаксичного дерева й не розв'язує імен, тому не встановлює, до якої таблиці належить колонка, хоча від цього залежить, наприклад, чи є дефектною неагрегована колонка. По-третє, метод виявлення не формалізовано, тобто не встановлено, як контекст, зокрема обмеження схеми `NOT NULL` і первинні ключі, обґрунтовує безпечність конструкції, тому критерій розмежування дефектної та безпечної конструкцій не визначено. Точність виявлення натомість підвищують перевіркою вибірки даних поточного стану бази, хоча висновок має бути чинним для будь-якого стану, сумісного зі схемою.

На відміну від SQLCheck, Brass і Goldberg [48] запропонували виявляти семантичні помилки в SQL-запитах лише за обмеженнями схеми для всіх допустимих станів бази. Частину перевірок реалізовано в навчальній системі DES [49, 50], зокрема без перевірки значень `NULL` у підзапитах `NOT IN`. Для аудиту еталонних запитів цей підхід має три обмеження. По-перше, формально обґрунтовано лише перевірку суперечливості умов, а решту ознак виявляють окремими евристиками без спільної моделі правил, і результат перевірки бінарний. По-друге, підхід орієнтовано на стандартний SQL. Неагреговані колонки поза `GROUP BY` автори вважають синтаксично недопустимими, тому такі запити не розглядають, тоді як SQLite, у форматі якого зберігаються бази Spider і BIRD, їх виконує і повертає значення з довільного рядка групи [51]. По-третє, підхід апробовано на навчальних запитах, і більшість ознак переліку автори відносять до надлишкових або неефективних формулювань, що не змінюють результату запиту.

Окремий клас засобів становлять лінтери SQL. Один із найпоширеніших відкритих лінтерів, SQLFluff [52], застосовує 79 правил до конкретного дерева розбору, побудованого за граматикою одного з 28 діалектів, зокрема

SQLite. На відміну від абстрактного синтаксичного дерева, воно зберігає пробіли й коментарі, потрібні для перевірки оформлення, але не зв'язує імена колонок із таблицями. Більшість правил стосується стилю й оформлення запиту, проте окремі правила виявляють конструкції, що впливають на результат, зокрема порівняння з `NULL` через рівність, `LIMIT` без `ORDER BY` і з'єднання таблиць без жодної умови. Правила SQLFluff не використовують схему бази даних, тому лінтер не може довести безпечність конструкції. Виявлення декартового добутку обмежено з'єднаннями без умови, тому з'єднання, умова якого посилається лише на одну таблицю, залишається без знахідки. Крім того, правила спираються на граматику діалекту, але не на його семантику. Наприклад, псевдонім `t` і посилання `T.label` лінтер вважає різними іменами, хоча в SQLite регістр імен не має значення. Результат перевірки також є бінарним.

Отже, відкритий інструмент sqlcheck обмежується поверхневим лексичним аналізом запиту, що призводить до хибних спрацювань, а методологія SQLCheck не має відкритої реалізації і для аудиту еталонних запитів обмежена наведеними недоліками. Підхід Brass і Goldberg не має спільної формальної моделі правил і стану, коли висновок неможливий, не враховує семантики діалекту СКБД, а його перелік ознак переважно стосується надлишкових формулювань у навчальних запитах. Лінтер SQLFluff не зв'язує імен колонок із таблицями, не використовує схему бази даних і не враховує семантики діалекту. Це зумовлює потребу в розробленні та програмній реалізації методу виявлення антипатернів в еталонних запитах наборів Text-to-SQL, який охоплює конструкції, що впливають на результат запиту, ґрунтується на формальній моделі правил над абстрактним синтаксичним деревом запиту, доводить безпечність конструкції на підставі обмежень схеми з урахуванням семантики діалекту СКБД для будь-якого сумісного зі схемою стану та відокремлює випадки, коли висновок неможливий. (Чернетка новизни. Вперше розроблено метод виявлення SQL-антипатернів в еталонних запитах наборів даних Text-to-SQL, який включає модель правила у вигляді трійки «вузли-кандидати абстрактного синтаксичного дерева — структурна умова — доказ зі схеми», алгоритм обчислення доказу за областями видимості дерева з урахуванням обмежень схеми, гарантій діалекту СКБД і умов самого запиту та функцію рішення з трьома статусами, а саме дефект, нерозв'язаний випадок

і коректний запит. На відміну від відомих засобів, де схема й дані лише евристично відфільтровують спрацювання або формально обґрунтовано лише перевірку суперечливості умов стандартного SQL з бінарним результатом, усі правила подано в єдиній моделі, статус «коректний» виводиться з обмежень схеми з урахуванням гарантій діалекту для будь-якого сумісного з нею стану бази, а статус «нерозв'язаний» відокремлює випадки, де доказ не завершено. Відбір класів за впливом на результат і діалектна критичність належать до пункту «удосконалено класифікацію».)

1.4.4 Детерміновані методи перевірки відповідності питання і SQL-запиту

Відсутність антипатернів у синтаксично правильному й виконуваному еталонному запиті не є достатньою умовою його відповідності питанню. Запит може містити хибне значення, межу порівняння, часову умову, напрямок сортування чи режим зіставлення рядків, а саме питання може містити лексичний дефект, що змінює його тлумачення. Детерміновані методи перевірки відповідності тексту й SQL-запиту запропоновано в суміжних задачах, а саме в оцінюванні тексту, згенерованого за запитом, і в перевірці запиту, згенерованого моделлю.

Метод BLEC [53] ґрунтується на правилах і призначений для автоматичного оцінювання тексту, згенерованого за логічною формою запиту. Він двобічно зіставляє ключові лексеми логічної форми, зокрема значення й оператори, з лексемами тексту. Правила методу автори зводять до трьох типів, а саме заперечення, оператори та правила, специфічні для окремого набору. Для кожної пари метод визначає бінарну мітку, а оцінку корпусу обчислює як частку пар без незіставлених лексем.

Метод PV-SQL [54] також ґрунтується на правилах, але застосовує їх у зворотному напрямку, до SQL-запиту, згенерованого за питанням. Він за шаблонами виділяє з питання та поля `evidence` обмеження десяти типів, а детермінований перевірник встановлює, чи містить згенерований SQL-запит потрібні конструкції. Перевірник вбудовано в цикл агента на основі мовної моделі, де повідомлення про порушення спрямовують виправлення запиту під час виведення. Для аудиту еталонних пар наборів Text-to-SQL методи BLEC і PV-SQL мають такі обмеження.

1. *Орієнтація на результати генерування.* Обидва методи оцінюють згенерований текст або згенерований SQL-запит, а збережені еталонні пари не перевіряють.
2. *Довіра до питання й додаткових відомостей.* PV-SQL трактує питання та поле `evidence` як безпомилкові джерела обмежень. Надійність перевірника автори оцінюють часткою еталонних запитів, що задовольняють витягнуті обмеження (99,39 %), тобто розбіжність пояснюють неточністю обмеження, а не дефектом анотації. Дефектів у самому питанні чи в додаткових відомостях жоден із методів не розглядає, а BLEC додаткових відомостей не використовує.
3. *Поверхневе подання запиту.* BLEC зіставляє лексеми із заздалегідь побудованого шаблону запиту без синтаксичного розбору. Перевірник PV-SQL, за описом авторів, встановлює наявність потрібних лексем або речень SQL у розборі аналізатора sqlparse, а у відкритій реалізації шукає підрядки в тексті запиту [55]. Жоден із методів не будує повного абстрактного синтаксичного дерева й не враховує діалекту СКБД, тому ознаку з питання неможливо пов'язати з конкретним предикатом чи областю видимості запиту. Зокрема, з правил для порівнянь, відбору top-k та екстремумів у відкритій реалізації PV-SQL залишилася лише перевірка наявності `LIMIT`, а оператори порівняння не перевіряються.
4. *Неповна простежуваність вердикту.* Знахідку не пов'язано водночас з ознакою питання й елементом запиту. Реалізація BLEC повертає незіставлені лексеми [56], а PV-SQL повертає текстові повідомлення про порушені обмеження без прив'язки до предиката чи колонки запиту.
5. *Бінарний результат*. Обидва методи не мають окремого статусу для випадків, коли правил недостатньо для висновку. Якщо шаблон не спрацював, PV-SQL обмеження не створює, а порожній перелік порушень трактується як успіх. Тому неможливість зробити висновок не відрізняється від підтвердженої відповідності.

Отже, наявні детерміновані методи перевірки відповідності питання й SQL-запиту орієнтовано на перевірку результатів генерування, а відповідність у них встановлюється за наявністю лексем чи підрядків без синтаксичного аналізу запиту. Це зумовлює потребу в розробленні та програмній реалізації детермінованого методу аудиту відповідності питання й еталонного

SQL-запиту в наборах Text-to-SQL. Метод має зіставляти ознаки, виділені з питання та додаткових відомостей, з типізованими елементами абстрактного синтаксичного дерева запиту з урахуванням діалекту СКБД, виявляти дефекти в запиті, питанні й додаткових відомостях, пов'язувати кожен вердикт з ознакою питання та елементом запиту, яких він стосується, і відокремлювати випадки, коли висновок неможливий. Серед розглянутих робіт такого методу не запропоновано.

1.4.5 Аудит на основі мовних моделей і гібридні методи

Перевірка семантичної відповідності пари «питання — SQL» потребує інтерпретації природної мови та предметних знань. Тому сучасні автоматизовані підходи використовують мовні моделі самостійно або поєднують їх із виконанням запитів, формальними перевірниками та експертним переглядом. Водночас аналіз цих підходів виявляє дві спільні методологічні особливості: залежність семантичного кроку від однієї мовної моделі та припущення про безпомилковість природномовного питання.

SQLDriller є автоматичним засобом виявлення й виправлення збережених пар Text-to-SQL (скоріше не програма аудиту) [34]. Навчальні розділи Spider і BIRD він обробляє суцільно й автоматично; для Spider Test і BIRD Dev автоматичне виправлення додатково пройшло повний ручний перегляд. SQLDriller автоматизує валідацію та виправлення анотацій на основі умови узгодженості за виконанням. Пара задовольняє цю умову на певному стані бази, якщо результат виконання SQL збігається з результатом, який логічно випливає з питання (автори зазначають, що ця умова є необхідною, але не достатньою). Гібридний перевірник на основі SQLSolver і VeriEQL будує невеликі синтетичні бази-контрприклади, які розрізняють еталонний запит і згенеровані кандидати. На ручно розмічених вибірках із 500 тренувальних пар SQLDriller виявив 81,4 % помилкових анотацій Spider і 76,1 % анотацій BIRD за частки хибнопозитивних спрацювань 11,0 % і 12,3 %, а повністю правильно виправив 60,1 % і 52,2 % помилкових пар.

Детальний аналіз архітектури SQLDriller виявляє два принципові обмеження. По-перше, кожний його генеративний та оцінювальний крок спирається на одну мовну модель без міжмодельного комітету. Десять SQL-кандидатів за один виклик породжує одна базова zero-shot модель;

якщо серед них немає правильного запиту, формальний перевірник не може побудувати потрібний контрприклад (це зумовлює більшість пропусків під час виправлення). Очікуваний результат питання на кожному контрприкладі визначає також одна модель (GPT-4) за більшістю з п'яти власних викликів. Таке голосування є перевіркою самоузгодженості однієї моделі: за систематичної помилки моделі у тлумаченні питання всі п'ять викликів дають хибний оракул, що за даними самих авторів є головним джерелом хибних звинувачень еталонних запитів. По-друге, SQLDriller припускає, що природномовне питання (та поле `evidence` у BIRD) є абсолютно правильним і однозначним оракулом, тому завжди перезаписує лише еталонний SQL. Неоднозначне питання він не редагує: модель виконує одне його читання, і за розбіжності з еталонним SQL пара може отримати інший запит. У навчальному розділі цей варіант не застосовують і лишають початковий SQL; на Spider Test і BIRD Dev таких пар 25 і 33, і під час оцінювання обидва запити зараховують як прийнятну відповідь. Якщо модель іншого читання не розглядає, неоднозначність лишається непоміченою. У підсумку вона зникає зі збереженої пари: збіг одного читання з еталонним SQL не лишає сліду, а відхилена пропозиція іншого SQL не позначає питання як неоднозначне. Шаблон породження цих кандидатів узято з C3 для Spider і з CHESS для BIRD [34, 57, 58]. Перед генерацією SQL обидва шаблони скорочують схему до таблиць і колонок, пов'язаних із формулюванням питання. C3 десять разів відбирає до чотирьох таблиць і лишає найчастіший набір, а потім по п'ять найчастіших колонок кожної таблиці, зберігаючи їхні зовнішні ключі. CHESS відсіює колонки, добирає таблиці й мінімальний набір колонок, але первинні та зовнішні ключі лишає навіть без семантичного зв'язку з питанням. Якщо питання не відповідає схемі, називає відсутню колонку, плутає таблицю або їй суперечить, відбір іде за цим формулюванням, і елемент схеми, який виявив би невідповідність, до генерації SQL може не дійти. Десять кандидатів будуються вже на скороченому фрагменті, тож запиту з відкинутою таблицею серед них немає. Далі GPT-4 виконує те саме питання, і SQLDriller або переписує еталонний SQL під це читання, або лишає початковий запит. Позначки, що питання не лягає на схему, інструмент не ставить. Проте в реальних наборах дефект часто лежить на боці питання: воно може містити механічну помилку в літералі порівняно зі збереженими

значеннями бази, спиратися на незадокументовану предметну домовленість або бути невдалим парафразом. Останнє особливо типово для набору Spider, у якому одному еталонному SQL-запиту зазвичай відповідає група з кількох перефразованих питань. Якщо в одному з питань під час перефразування пропущено ключове слово, змінено назву атрибута або інвертовано умову порівняння, тоді як сусіднє питання з тим самим SQL точно відображає задум авторів, ізольований аудит у SQLDriller сприймає дефектне формулювання за істину й переписує правильний еталонний SQL під помилкове питання.

Jin та ін. запропонували дослідницького агента SAR-Agent і людино-машинну процедуру SAPAR [59]. SAR-Agent працює на основі однієї мовної моделі (OpenAI o3) у багатоходовому режимі (до 30 кроків), виконує допоміжні запити до бази даних і формує діагностичний звіт із поясненням та чорновим варіантом виправленого SQL. Сам агент не ухвалює остаточного рішення: у процедурі SAPAR кожне зауваження розглядають анотатори й експерти з SQL. На відміну від чисто SQL-орієнтованих методів, автори SAPAR враховують, що дефект може міститися в самому питанні, полі `evidence` або даних (зокрема, під час ручного виправлення 100 пар BIRD Dev у 48 проблемних прикладах експерти змінили 41 SQL-запит, 19 питань, 17 полів `evidence` і наповнення 6 баз). Проте виправлення питань і додаткових відомостей у SAPAR виконується вручну людьми, а запропонований агентом SQL-запит автори навіть не оцінюють кількісно, оскільки без попередньої корекції питання заміна лише SQL є ненадійною. Із залученням додаткового ручного пошуку автори оцінили частку помилкових анотацій у 52,8 % для BIRD Mini-Dev і 62,8 % для Spider 2.0-Snow та показали, що на виправленій підвибірці зі 100 пар BIRD Dev рангова кореляція Спірмена з початковим рейтингом 16 агентів падає з 0,85 до 0,32. Отже, SAR-Agent є інтерактивним помічником експерта на базі однієї моделі, який не забезпечує автономного детермінованого аудиту, не враховує зв’язків між перефразованими питаннями корпусу й потребує значних обчислювальних і експертних витрат.

Alrashed та ін. розглядають верифікацію передбачень без доступу до еталонної мітки за двома сигналами однієї базової моделі (Gemini 2.5 Pro): зворотним перекладом SQL у текст із критикою та перевіркою узгодженості виконання на невеликих синтетичних базах, згенерованих самою моделлю [60]. Серед еталонних запитів, відхилених другим верифікатором, авто-

ри за вибірками по 30 прикладів оцінили нижню межу кількості помилкових SQL-анотацій (близько 349 для BIRD, 290 для Spider і 35 для KaggleDBQA). Як і в SQLDriller, питання тут вважається еталоном наміру, сусідні парафрази не аналізуються, а приклади, які верифікатор прийняв, залишаються поза перевіркою.

Підхід SALUS поєднує чотирьох LLM-агентів, 44 функції розмітки, слабкий нагляд і класифікатор на ознаках еталонного SQL [61]. На вибірці з 298 прикладів BIRD Dev метод досяг $F_1 = 0{,}9194$, а після калібрування автори оцінили частку помилкових анотацій приблизно у 37 % для BIRD Dev і 27 % для Spider Dev. Використання кількох агентів зменшує залежність від однієї моделі, проте метод формує лише бінарну мітку щодо пари, не виконує автоматичного виправлення чи розмежування дефекту між питанням і SQL у групах парафраз, не перевіряє посилальної цілісності баз і діалектних антипатернів та не досліджує тренувальні розділи наборів.

Загалом опора більшості розглянутих методів на одну мовну модель робить аудит вразливим до її індивідуальних зміщень за позицією, довжиною та на користь власних генерацій [62]. Застосування комітету різнорідних моделей робить розбіжності між оцінювачами явними й підвищує узгодженість з експертами [63], проте і його висновок доцільно розглядати як зважений сигнал поряд із детермінованими перевірками, а не як безумовну еталонну мітку.

Отже, методи на основі мовних моделей дозволяють оцінювати семантику пари, недоступну суто синтаксичним правилам, але наявні рішення або зводять семантичний оракул до однієї моделі (SQLDriller, SAR-Agent, Alrashed та ін.), або покладають локалізацію й виправлення дефектів на ручну працю експертів (SAPAR). Однобічне припущення про безпомилковість питання без аналізу фактичних значень бази даних і парафраз із тим самим SQL призводить до хибної заміни коректних SQL-запитів під дефектні формулювання питань.

1.4.6 Порівняльний аналіз засобів аудиту

Розглянуті засоби зіставлено за шістьма критеріями. Критерії К1–К3 відповідають предметам перевірки без участі питання: цілісності схеми й даних (К1), розбору та виконанню еталонного SQL (К2) і антипатернам еталонно-

го SQL (К3). Критерій К4 означає перевірку відповідності питання еталонному SQL. Критерій К5 вказує, що засіб аудитує збережені еталонні анотації, а не лише передбачення моделі або агрегований профіль набору. Критерій К6 означає детермінованість рішення. Результати зіставлення наведено в табл. 1.2.

Таблиця 1.2 – Порівняння засобів контролю якості анотацій Text-to-SQL

Засіб	К1	К2	К3	К4	К5	К6	Рішення ухвалює
Ручний аудит [37, 38]	△	△	△	✓	✓	×	експерт
Mitsopoulou, Koutrika [26]	△	△	×	×	×	✓	статистика профілю
SQLCheck [47]	×	△	✓	×	△	✓	правила
Профілювання схеми [39–41]	✓	×	×	×	×	✓	правила
Pandey та ін. [44]	△	✓	×	△	×	—	концептуальна схема
BLEC [53]	×	△	×	△	×	✓	правила
PV-SQL [54]	×	△	×	△	×	△	правила, мовна модель
SQLDriller [34]	×	✓	△	✓	✓	×	мовна модель, SMT
Jin та ін. [59]	×	△	×	✓	✓	×	мовна модель, експерт
Alrashed та ін. [60]	×	✓	×	✓	△	×	мовна модель, виконання
SALUS [61]	×	△	△	✓	✓	×	мовні моделі, слабкий нагляд

К1 — цілісність схеми і даних; К2 — розбір і виконання еталонного SQL; К3 — антипатерни еталонного SQL; К4 — відповідність питання й еталонного SQL; К5 — аудит збереженої еталонної анотації; К6 — детерміноване рішення. ✓ — повне охоплення, △ — часткове або непряме, × — відсутнє, прочерк — не визначено.

Табл. 1.2 показує, що детерміновані засоби й методи на основі мовних моделей доповнюють одні одних. Перші відтворювано характеризують SQL, схему або окремі формальні зв’язки, але не встановлюють загальної відповідності збереженої еталонної пари. Другі безпосередньо оцінюють семантику пари, однак успадковують помилки й недетермінованість мовної моделі.

Розбіжність висновків особливо помітна на BIRD Dev. Частка помилкових анотацій за SALUS після калібрування становить приблизно 37 %, а за SQLDriller після автоматичного виправлення і повного ручного перегляду — 50,1 % [34, 61]. Ці числа не можна безпосередньо порівнювати як точність методів, оскільки роботи використовують різні версії набору, критерії помилки

й процедури відбору. Проте сама розбіжність свідчить, що бінарна мітка без типізованого доказу й зафіксованих критеріїв недостатня для відтворюваного аудиту.

Жоден із розглянутих підходів не охоплює К1–К5 одночасно й не поєднує їх із детермінованим ядром. Саме сукупність цих обмежень визначає наукову задачу, формулювання якої наведено в наступному підрозділі.

1.5 Постановка наукових завдань дисертаційного дослідження

Проведений аналіз дав змогу виявити такі невирішені проблеми забезпечення якості наборів даних Text-to-SQL:

1. Відсутність методу виявлення антипатернів в еталонних SQL-запитах з урахуванням діалекту СКБД і обмежень схеми бази даних. Такий метод має охоплювати класи, що впливають на результат еталонного запиту, аналізувати абстрактне синтаксичне дерево запиту, визначати критичність знахідки залежно від семантики діалекту, використовувати обмеження схеми як достатню ознаку безпечності конструкції, відокремлювати випадки, коли свідчень для висновку недостатньо, та формувати порівнювані профілі дефектів різних наборів.

2. Відсутність детермінованого методу аудиту збережених пар «питання — еталонний SQL» на рівні всього набору. Кожне рішення має спиратися на локалізоване свідчення з питання, додаткових відомостей або SQL AST. За недостатності чи неоднозначності свідчень метод повинен утримуватися від бінарного висновку, а повторюваність недокументованого шаблону в корпусі не повинна вважатися доказом його правильності.

3. Відсутність інформаційної технології багатовимірного аудиту, яка об’єднує перевірку цілісності схеми й даних, синтаксичний аналіз і виконання еталонних запитів, виявлення антипатернів та перевірку відповідності питання SQL-запиту. Вона має зберігати знахідки з різних вимірів окремо, прив’язувати їх до конкретних анотацій і формувати пріоритет перевірки. На відміну від бінарного аудитора, такий профіль дає змогу встановити джерело дефекту й обґрунтовано вибрати спосіб виправлення.

4. Відсутність оцінювання впливу виправлених анотацій на параметрично ефективну адаптацію після такого аудиту. Розглянуті в підрозділі 1.3.4 експерименти охоплюють повне донавчання, SFT із LoRA і навчання з підкріпленням із LoRA, але не QLoRA-SFT після багатовимірного аудиту. Не встановлено також, як на результат окремо впливають виправлення тренувальної вибірки, виправлення валідаційної вибірки й усунення різних типів дефектів.

Сукупність зазначених проблем визначає наукову задачу дисертаційної роботи: розроблення методів і засобів забезпечення якості наборів даних Text-to-SQL, які охоплюють діалектний аудит еталонних SQL-запитів, детерміновану перевірку узгодженості питання й еталонного SQL, багатовимірний аудит анотацій і оцінювання впливу виправлень на параметрично ефективну адаптацію мовних моделей.

Для розв'язання поставленої наукової задачі необхідно виконати такі завдання:

1. Проаналізувати задачу Text-to-SQL, набори даних, метрики оцінювання, методи адаптації мовних моделей і засоби аудиту анотацій та обґрунтувати невирішені проблеми забезпечення якості наборів.

2. Розробити метод виявлення антипатернів в еталонних SQL-запитах наборів даних Text-to-SQL з урахуванням діалекту СКБД та обмежень схеми бази даних.

3. Розробити детермінований метод аудиту пар «питання — еталонний SQL» із локалізованими свідченнями та явним утриманням від рішення за їх недостатності.

4. Розробити інформаційну технологію багатовимірного аудиту наборів даних Text-to-SQL, яка формує типізований профіль дефектів кожної анотації.

5. Дослідити вплив виправлених анотацій на QLoRA-SFT, порівнявши навчання на початкових і виправлених тренувальних та валідаційних вибірках і розмежувавши внесок різних типів виправлень.

Перше завдання розв'язано в межах цього розділу. Завдання 2 є предметом розділу 2, завдання 3 — розділу 3. Завдання 4 і 5 розв'язуються в розділі 4, де побудовано інформаційну технологію багатовимірного аудиту та оцінено вплив виправлень на QLoRA-SFT.

Висновки до розділу 1

У розділі формалізовано задачу Text-to-SQL як відображення питання, схеми, стану бази даних і додаткових відомостей у SQL-запит. Показано, що exact match, точність виконання та тестовий набір станів оцінюють різні властивості передбачення, але жодна з цих метрик не доводить правильності самої еталонної анотації. WikiSQL, Spider і BIRD відрізняються структурою схем, складністю запитів і наявністю додаткових відомостей, а синтетичні корпуси Gretel і SynSQL-2.5M забезпечують більший масштаб ціною автоматизованого контролю якості.

Проаналізовано навчання в контексті, повне донавчання, LoRA та QLoRA. Незалежно від кількості оновлюваних параметрів, SFT збільшує ймовірність еталонної послідовності, тому дефектний SQL формує неправильний навчальний сигнал. Дослідження з повним донавчанням і LoRA підтверджують вплив виправлених анотацій, проте не встановлюють ефекту окремих типів дефектів під час SFT із QLoRA та не розмежовують наслідки виправлення тренувальної і валідаційної вибірок.

Порівняння ручних, детермінованих і модельних засобів аудиту виявило фрагментарність наявних рішень. Ручна перевірка враховує предметну семантику, але потребує значних витрат. Детерміновані засоби відтворювано аналізують окремі властивості SQL, схем або тексту, проте не формують цілісного рішення щодо збереженої еталонної пари. SQLDriller, SALUS та SAPAR оцінюють семантику пари, але їхній результат залежить від мовної моделі, експерта або обох. Жоден розглянутий підхід не охоплює одночасно цілісність бази даних, властивості еталонного SQL і відповідність питання SQL-запиту.

На підставі встановлених обмежень сформульовано чотири проблеми, які потребують вирішення. На підставі проведеного аналізу сформульовано наукову задачу дисертаційної роботи — розроблення методів і засобів забезпечення якості наборів даних Text-to-SQL — та визначено п'ять завдань для

її розв'язання.

Список використаних джерел до розділу 1

[1] Liang Shi, Zhengju Tang, Nan Zhang, Xiaotong Zhang, and Zhi Yang. A survey on employing large language models for Text-to-SQL tasks. *ACM Computing Surveys*, 58(2):1–37, 2025. doi: 10.1145/3737873.

[2] Yuren Mao, Yuhang Ge, Yijiang Fan, Wenyi Xu, Yu Mi, Zhonghao Hu, and Yunjun Gao. A survey on LoRA of large language models. *Frontiers of Computer Science*, 19:197605, 2025. doi: 10.1007/s11704-024-40663-9.

[3] Ali Mohammadjafari, Anthony S. Maida, and Raju Gottumukkala. From natural language to SQL: Review of LLM-based Text-to-SQL systems, 2024.

[4] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, Zilin Zhang, and Dragomir Radev. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and Text-to-SQL task. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 3911–3921. Association for Computational Linguistics, 2018. doi: 10.18653/v1/D18-1425.

[5] Jinyang Li, Binyuan Hui, Ge Qu, Jiaxi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, Xuanhe Zhou, Chenhao Ma, Guoliang Li, Kevin C. C. Chang, Fei Huang, Reynold Cheng, and Yongbin Li. Can LLM already serve as a database interface? A big bench for large-scale database grounded Text-to-SQLs. In *Advances in Neural Information Processing Systems 36, Datasets and Benchmarks Track*, pages 42330–42357, 2023.

[6] Tao Yu, Rui Zhang, Heyang Er, Suyi Li, Eric Xue, Bo Pang, Xi Victoria Lin, Yi Chern Tan, Tianze Shi, Zihan Li, Youxuan Jiang, Michihiro Yasunaga, Sungrok Shim, Tao Chen, Alexander Fabbri, Zifan Li, Luyao Chen, Yuwen Zhang, Shreya Dixit, Vincent Zhang, Caiming Xiong, Richard Socher, Walter Lasecki, and Dragomir Radev. CoSQL: A conversational Text-to-SQL challenge towards cross-domain natural language interfaces to databases. In *Proceedings of the 2019 Conference on Empirical Methods in Natural*

Language Processing and the 9th International Joint Conference on Natural Language Processing, pages 1962–1979, 2019. doi: 10.18653/v1/D19-1204.

[7] Xinyu Liu, Shuyu Shen, Boyan Li, Peixian Ma, Runzhi Jiang, Yuxin Zhang, Ju Fan, Guoliang Li, Nan Tang, and Yuyu Luo. A survey of Text-to-SQL in the era of LLMs: Where are we, and where are we going? *IEEE Transactions on Knowledge and Data Engineering*, 37(10):5735–5754, 2025. doi: 10.1109/TKDE.2025.3592032.

[8] W. A. Woods, R. M. Kaplan, and B. Nash-Webber. The lunar sciences natural language information system: Final report. Technical Report 2378, Bolt Beranek and Newman Inc., June 1972. URL https://brenocon.com/Woods1972Lunar.pdf.

[9] Yunyao Li, Huahai Yang, and H. V. Jagadish. NaLIX: An interactive natural language interface for querying XML. In *Proceedings of the 2005 ACM SIGMOD International Conference on Management of Data*, 2005. URL https://yunyaoli.github.io/files/sigmod05-nalix.pdf.

[10] Fei Li and H. V. Jagadish. Constructing an interactive natural language interface for relational databases. *Proceedings of the VLDB Endowment*, 8 (1):73–84, 2014. URL https://www.vldb.org/pvldb/vol8/p73-li.pdf.

[11] Li Dong and Mirella Lapata. Language to logical form with neural attention. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 33–43, 2016. doi: 10.18653/v1/P16-1004.

[12] Victor Zhong, Caiming Xiong, and Richard Socher. Seq2SQL: Generating structured queries from natural language using reinforcement learning. *CoRR*, abs/1709.00103, 2017. doi: 10.48550/arXiv.1709.00103.

[13] Xiaojun Xu, Chang Liu, and Dawn Song. SQLNet: Generating structured queries from natural language without reinforcement learning, 2017.

[14] Bailin Wang, Richard Shin, Xiaodong Liu, Oleksandr Polozov, and Matthew Richardson. RAT-SQL: Relation-aware schema encoding and linking for Text-to-SQL parsers. In *Proceedings of the 58th Annual Meeting of the*

Association for Computational Linguistics, pages 7567–7578, 2020. doi: 10.18653/v1/2020.acl-main.677.

[15] Pengcheng Yin, Graham Neubig, Wen-tau Yih, and Sebastian Riedel. TaBERT: Pretraining for joint understanding of textual and tabular data. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 8413–8426, 2020. doi: 10.18653/v1/2020.acl-main.745.

[16] Torsten Scholak, Nathan Schucher, and Dzmitry Bahdanau. PICARD: Parsing incrementally for constrained auto-regressive decoding from language models. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 9895–9901, 2021. doi: 10.18653/v1/2021.emnlp-main.779.

[17] Nitarshan Rajkumar, Raymond Li, and Dzmitry Bahdanau. Evaluating the Text-to-SQL capabilities of large language models, 2022.

[18] Mohammadreza Pourreza and Davood Rafiei. DIN-SQL: Decomposed in-context learning of Text-to-SQL with self-correction. In *Advances in Neural Information Processing Systems 36*, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/72223cc66f63ca1aa59edaec1b3670e6-Abstract.html.

[19] Dawei Gao, Haibin Wang, Yaliang Li, Xiuyu Sun, Yichen Qian, Bolin Ding, and Jingren Zhou. Text-to-SQL empowered by large language models: A benchmark evaluation. *Proceedings of the VLDB Endowment*, 17(5):1132–1145, 2024. doi: 10.14778/3641204.3641221.

[20] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *Proceedings of the 10th International Conference on Learning Representations*, 2022.

[21] Haoyang Li, Shang Wu, Xiaokang Zhang, Xinmei Huang, Jing Zhang, Fuxin Jiang, Shuai Wang, Tieying Zhang, Jianjun Chen, Rui Shi, Hong Chen, and Cuiping Li. OmniSQL: Synthesizing high-quality Text-to-SQL data at scale. *Proceedings of the VLDB Endowment*, 18(11):4695–4709, 2025. doi: 10.14778/3749646.3749723.

[22] Gretel.ai. Synthetic Text-to-SQL dataset. Hugging Face Datasets, 2024. URL https://huggingface.co/datasets/gretelai/synthetic_text_to_sql.

[23] Shuaichen Chang, Jun Wang, Mingwen Dong, Lin Pan, Henghui Zhu, Alexander Hanbo Li, Wuwei Lan, Sheng Zhang, Jiarong Jiang, Joseph Lilien, Steve Ash, William Yang Wang, Zhiguo Wang, Vittorio Castelli, Patrick Ng, and Bing Xiang. Dr.Spider: A diagnostic evaluation benchmark towards Text-to-SQL robustness. In *Proceedings of the 11th International Conference on Learning Representations*, 2023.

[24] Shuaichen Chang, Pengfei Liu, Yun Tang, Jing Huang, Xiaodong He, and Bowen Zhou. Zero-shot Text-to-SQL learning with auxiliary task. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 7488–7495, 2020. doi: 10.1609/aaai.v34i05.6246.

[25] Salesforce. WikiSQL dataset. Hugging Face Datasets, 2017. URL https://huggingface.co/datasets/Salesforce/wikisql.

[26] Anna Mitsopoulou and Georgia Koutrika. Analysis of Text-to-SQL benchmarks: Limitations, challenges and opportunities. In *Proceedings of the 28th International Conference on Extending Database Technology*, pages 199–212, 2025. doi: 10.48786/EDBT.2025.16.

[27] Catherine Finegan-Dollak, Jonathan K. Kummerfeld, Li Zhang, Karthik Ramanathan, Sesh Sadasivam, Rui Zhang, and Dragomir Radev. Improving Text-to-SQL evaluation methodology. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 351–360, 2018. doi: 10.18653/v1/P18-1033.

[28] Ruiqi Zhong, Tao Yu, and Dan Klein. Semantic evaluation for Text-to-SQL with distilled test suites. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, pages 396–411. Association for Computational Linguistics, 2020. doi: 10.18653/v1/2020.emnlp-main.29.

[29] Zijin Hong, Zheng Yuan, Qinggang Zhang, Hao Chen, Junnan Dong, Feiran Huang, and Xiao Huang. Next-generation database interfaces: A survey of LLM-based Text-to-SQL, 2025.

[30] Haoyang Li, Jing Zhang, Hanbing Liu, Ju Fan, Xiaokang Zhang, Jun Zhu, Renjie Wei, Hongyan Pan, Cuiping Li, and Hong Chen. CodeS: Towards building open-source language models for Text-to-SQL. *Proceedings of the ACM on Management of Data*, 2(3):1–28, 2024. doi: 10.1145/3654930.

[31] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems 36*, 2023.

[32] Dan Biderman, Jacob Portes, Jose Javier Gonzalez Ortiz, Mansheej Paul, Philip Greengard, Connor Jennings, Daniel King, Sam Havens, Vitaliy Chiley, Jonathan Frankle, Cody Blakeney, and John P. Cunningham. LoRA learns less and forgets less. *Transactions on Machine Learning Research*, 2024. ISSN 2835-8856. URL https://openreview.net/forum?id=aloEru2qCG.

[33] Zhewei Yao, Guoheng Sun, Łukasz Borchmann, Zheyu Shen, Minghang Deng, Bohan Zhai, Hao Zhang, Ang Li, and Yuxiong He. Arctic-Text2SQL-R1: Simple rewards, strong reasoning in Text-to-SQL. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 26966–26995, 2026. doi: 10.18653/v1/2026.findings-acl.1345.

[34] Yicun Yang, Zhaoguo Wang, Yu Xia, Zhuoran Wei, Haoran Ding, Ruzica Piskac, Haibo Chen, and Jinyang Li. Automated validating and fixing of Text-to-SQL translation with execution consistency. *Proceedings of the ACM on Management of Data*, 3(3):1–28, 2025. doi: 10.1145/3725271.

[35] Yuanzhen Xie, Liu Ye, Jiqun Chu, Mochi Gao, Hehuan Liu, Yunzhi Tan, Bo Hu, and Zang Li. DCMM-SQL: Automated data-centric pipeline and multi-model collaboration training for Text-to-SQL model, 2025.

[36] Yuxuan Zhu and Daniel Kang. Noisy data is destructive to reinforcement learning with verifiable rewards, 2026.

[37] Mohammadreza Pourreza and Davood Rafiei. Evaluating cross-domain Text-to-SQL models and benchmarks. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 1601–1611, 2023. doi: 10.18653/v1/2023.emnlp-main.99.

[38] Niklas Wretblad, Fredrik Riseby, Rahul Biswas, Amin Ahmadi, and Oskar Holmström. Understanding the effects of noise in Text-to-SQL: An examination of the BIRD-Bench benchmark. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 356–369, 2024. doi: 10.18653/v1/2024.acl-short.34.

[39] Ziawasch Abedjan, Lukasz Golab, and Felix Naumann. Profiling relational data: A survey. *The VLDB Journal*, 24(4):557–581, 2015. doi: 10.1007/s00778-015-0389-y.

[40] Sualeh Fatehi. SchemaCrawler: Database schema discovery and comprehension tool. https://www.schemacrawler.com/, 2026.

[41] Mozhgan Memari, Sebastian Link, and Gillian Dobbie. SQL data profiling of foreign keys. In *Proceedings of the 34th International Conference on Conceptual Modeling*, pages 229–243, 2015. doi: 10.1007/978-3-319-25264-3_17.

[42] SQLite. SQLite foreign key support. https://sqlite.org/foreignkeys.html, 2025.

[43] Jiawei Shen, Chengcheng Wan, Ruoyi Qiao, Jiazhen Zou, Hang Xu, Yuchen Shao, Yueling Zhang, Weikai Miao, and Geguang Pu. Understanding, detecting, and repairing real-world in-context-learning-based text-to-sql errors. *Proceedings of the ACM on Software Engineering*, 3(FSE):3722–3745, 2026. doi: 10.1145/3808171.

[44] P. Pandey, D. Patel, S. Mandvikar, and N. Kota. Ensuring data accuracy in Text-to-SQL systems: A comprehensive validation framework. *International Journal of Computer Trends and Technology*, 72(12):17–24, 2024. doi: 10.14445/22312803/IJCTT-V72I12P103.

[45] Bill Karwin. *SQL Antipatterns: Avoiding the Pitfalls of Database Programming*. Pragmatic Bookshelf, 2010.

[46] Joy Arulraj et al. SQLCheck: Automatically identify anti-patterns in SQL queries, version 1.3. https://github.com/jarulraj/sqlcheck, 2024.

[47] Prashanth Dintyala, Arpit Narechania, and Joy Arulraj. SQLCheck: Automated detection and diagnosis of SQL anti-patterns. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, pages 2331–2345, 2020. doi: 10.1145/3318464.3389754.

[48] Stefan Brass and Christian Goldberg. Semantic errors in SQL queries: A quite complete list. *Journal of Systems and Software*, 79(5):630–644, 2006. doi: 10.1016/j.jss.2005.06.028.

[49] Fernando Sáenz-Pérez. Applying constraint logic programming to SQL semantic analysis. *Theory and Practice of Logic Programming*, 19(5–6):808–825, 2019.

[50] Fernando Sáenz-Pérez. DES: Datalog educational system, version 6.9. https://des.sourceforge.io/, 2025.

[51] SQLite. SELECT: Bare columns in an aggregate query. https://sqlite.org/lang_select.html#bare_columns_in_an_aggregate_query, 2025.

[52] SQLFluff contributors. SQLFluff: A modular SQL linter and auto-formatter with support for multiple dialects, version 4.1.0. https://github.com/sqlfluff/sqlfluff, 2026.

[53] Chang Shu, Yusen Zhang, Xiangyu Dong, Peng Shi, Tao Yu, and Rui Zhang. Logic-consistency text generation from semantic parses. In *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*, pages 4414–4426, 2021. doi: 10.18653/v1/2021.findings-acl.388.

[54] Yuan Tian and Tian Zhang. PV-SQL: Synergizing database probing and rule-based verification for Text-to-SQL agents. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 25827–25845, 2026. doi: 10.18653/v1/2026.findings-acl.1286.

[55] Yuan Tian and Tian Zhang. PV-SQL: Synergizing database probing and rule-based verification for Text-to-SQL agents, source code. https://github.com/magic-YuanTian/PV-SQL, 2026.

[56] Chang Shu et al. BLEC: Bidirectional logic evaluation of consistency, source code. https://github.com/chatc/BLEC, 2021.

[57] Xuemei Dong, Chao Zhang, Yuhang Ge, Yuren Mao, Yunjun Gao, Lu Chen, Jinshu Lin, and Dongfang Lou. C3: Zero-shot Text-to-SQL with ChatGPT, 2023.

[58] Shayan Talaei, Mohammadreza Pourreza, Yu-Chen Chang, Azalia Mirhoseini, and Amin Saberi. CHESS: Contextual harnessing for efficient SQL synthesis, 2024.

[59] Tengjun Jin, Yoojin Choi, Yuxuan Zhu, and Daniel Kang. Pervasive annotation errors break Text-to-SQL benchmarks and leaderboards. *Proceedings of the VLDB Endowment*, 19(5):931–944, 2026. doi: 10.14778/3796195.3796206.

[60] Tarfah Alrashed, Madhup Sukoon, David R. Karger, and Natasha Noy. Developing and benchmarking verification algorithms to improve Text-to-SQL generation. *Proceedings of the VLDB Endowment*, 19(11):3732–3744, 2026. doi: 10.14778/3836663.3836721.

[61] Shiyuan Zhou, Ashwin Gerard Colaco, Sainyam Galhotra, and Sharad Mehrotra. SALUS: Automated auditing of NL-to-SQL benchmarks through weak supervision of multi-agent output, 2026.

[62] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems 36*, 2023.

[63] Chi-Min Chan, Weize Chen, Yusheng Su, Jianxuan Yu, Wei Xue, Shanghang Zhang, Jie Fu, and Zhiyuan Liu. ChatEval: Towards better LLM-based evaluators through multi-agent debate. In *Proceedings of the 12th International Conference on Learning Representations*, 2024.